Studio Nx

Backend(FastAPI/Python) 27 — create_refresh_token()

2026-10-12

Backend(FastAPI/Python) 27 — `create_refresh_token()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-05 로그인: Refresh Token 발급 (PRD 01 FR-105, FR-109)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	26 `create_access_token()` · 28 `decode_refresh_token()`

1. 하는 일

회원 번호로 **3일짜리 Refresh Token(JWT)**을 만듭니다. 26번과 모양이 같고, 종류와 수명만 다릅니다.

	Access Token (26)	Refresh Token (이 함수)
`type`	`"access"`	`"refresh"`
수명	30분	**3일** (`jwt_refresh_days`)
브라우저에 두는 곳	JS 메모리 (`api.js` 변수)	**HttpOnly 쿠키 `snx_refresh`**입니다. JS가 읽을 수 없습니다
서버로 가는 경로	모든 API 요청의 `Authorization: Bearer …`	`/api/v1/auth/…` 요청에만 쿠키로 자동으로 실립니다
쓰임	API 요청	새 Access Token을 받을 때 씁니다 (33 `/auth/refresh`)

2. 언제 쓰나

쓰는 곳	하는 일
P1-05 `POST /auth/login` (32)	로그인에 성공하면 이 토큰을 쿠키에 담아 응답합니다 (31 `set_refresh_cookie`)

두 토큰이 함께 움직이는 순서

순서	일어나는 일
①	로그인하면 Access(30분)는 응답 본문으로, Refresh(3일)는 쿠키로 받습니다
②	30분 동안의 API 요청은 Access로 보냅니다
③	Access가 만료되면 서버가 401로 응답하고, 브라우저가 `POST /auth/refresh`를 보냅니다 (쿠키가 자동으로 실립니다)
④	서버가 Refresh를 검사하고(28) 새 Access를 만들어 줍니다(26)
⑤	3일이 지나면 Refresh도 만료되어 다시 로그인해야 합니다

자동 로그인 체크와의 관계: 토큰의 수명은 **항상 3일**입니다. 체크 여부는 **쿠키**의 수명만 바꿉니다 (31).

로그인 모달의 "Keep me signed in"	쿠키	결과
체크함	3일짜리 쿠키	브라우저를 닫았다가 다시 열어도 3일 동안 로그인이 유지됩니다
체크하지 않음	세션 쿠키 (수명 없음)	브라우저를 닫으면 쿠키가 사라져서 다시 로그인해야 합니다

3. 코드

import (파일 맨 위)

```
import re
import string
from datetime import UTC, datetime, timedelta

import bcrypt
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import입니다 (26번과 같고, 이번에 새로 추가한 것은 없습니다). 이 함수는 **`timedelta`와 `get_settings`**를 쓰고, 토큰을 만드는 일은 26번의 `_create_token()`에 맡깁니다.

함수

```
def create_refresh_token(user_id: int) -> str:
    days = get_settings().jwt_refresh_days
    return _create_token(user_id, "refresh", timedelta(days=days))
```

문법 풀이

import 문법	뜻
`from datetime import timedelta`	기간을 나타내는 타입입니다 (26번에서 이미 가져왔습니다)

식	뜻
`get_settings().jwt_refresh_days`	`.env` 설정입니다. 기본값은 3입니다 (`core/config.py`)
`timedelta(days=3)`	3일, 즉 259,200초입니다
`_create_token(user_id, "refresh", …)`	26번과 같은 함수에 종류와 수명만 다르게 넘깁니다

4. 흐름

1. 설정에서 수명(3일)을 읽습니다.
2. 26번의 `_create_token()`으로 `type: "refresh"` 토큰을 만듭니다.

5. 왜 이렇게

선택	이유
Access 토큰과 나눔	자주 오가는 Access는 짧게(30분) 두고, 오래 가는 Refresh는 JS가 읽을 수 없는 쿠키에 숨깁니다. 스크립트 공격(XSS)으로 Access를 훔치더라도 30분이 지나면 쓸 수 없습니다
`type: "refresh"`	05번은 Access 자리에서 이 토큰을 거절하고, 28번은 반대로 Access 토큰을 거절합니다. 그래서 두 토큰을 서로 바꿔 쓸 수 없습니다
토큰 수명은 체크 여부와 상관없이 3일	쿠키가 브라우저에서 사라지면 토큰도 쓸 일이 없습니다. 그래서 토큰을 두 가지로 만들 필요가 없습니다

선택	이유
같은 비밀값과 알고리즘	26번과 같은 `_create_token`을 써서 서명 방식이 하나로 유지됩니다

6. 시험 (`tests/test_security.py`에 3개를 추가해서 모두 17개)

시험	확인한 것
내용	`sub`는 `"7"`, `type`은 `"refresh"`, `exp - iat`는 259,200초(3일)입니다
Access 자리에 쓰기	05번 `decode_access_token`이 401로 응답합니다. 3일짜리 토큰으로 API를 부를 수 없습니다
두 토큰은 다름	같은 회원의 Access와 Refresh가 서로 다릅니다

전체 시험 194개가 통과했고, `ruff` 검사도 통과했습니다.